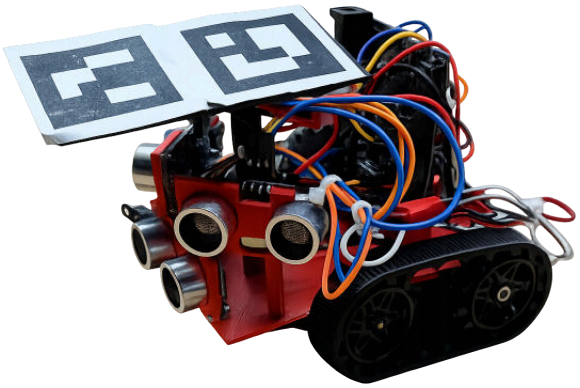


Final Report

Ava Alteveer (Mechanical Engineering), Amresh Balakrishnan (Computer Engineering), & Abuzer Gerdek (Computer Engineering)

ENGR-122 Section A1 Group #2

5/8/2026

I pledge my honor that I have abided by the Stevens Honor System.

Electronically Signed: Amresh, Ava, Abuzer

Executive Summary (Balakrishnan, Gerdek, Alteveer)

- ➔ Our final design featured an autonomous robot that successfully navigated the Smart Area using an MQTT-based coordinate system, ArUco markers, and ultrasonic sensors, in order to reach four assigned targets in order, stop for two seconds at each target and avoid walls and obstacles. The robot featured an OLED display, a custom 3-D printed body, no glue or tape, no breadboard, a custom circuit, and a slide switch.
- ➔ The final prototype was built using a Zumo chassis, WeMos D1 microcontroller, motor controller, three HC-SR04 ultrasonic sensors, battery pack, custom 3-D printed mounts, and a separate ArUco marker stand. The electrical system used a lever style connector and a wired power bus for more reliable power distribution.
- ➔ The Arduino C++ software combined MQTT position tracking, a math.h library, and object oriented programming principles to implement a path finding algorithm and obstacle avoidance algorithm. The robot calculated distance and angle error from the target and robot coordinate values, adjusted motor speeds to correct its heading, and used ultrasonic readings to avoid nearby obstacles.
- ➔ All things considered, our robot successfully integrated mechanical design, electrical power distribution, sensor feedback, and autonomous navigation algorithms into a functional system to complete the design challenge.

Introduction (Balakrishnan)

- ➔ This project required the development of an autonomous robot, modeled after the real world use case of a robotic tour guide – our final system was tasked with following a specified path while avoiding obstacle collisions. The system was designed around three major components: mechanical structure, electrical hardware, and software algorithms. Each subsystem had to work alongside the other subsystems so that the robot could receive position data, detect obstacles, control motor motion, and operate reliably during testing.
- ➔ The mechanical platform was based on a Zumo chassis with two tread-driven motors. Custom parts were modeled in Solidworks and fabricated through 3D printing using Flashprint. These parts included the main electronics mounting structure, ultrasonic sensor mounts, and a separate holder for the ArUco marker. The ArUco markers were necessary for the overhead camera system, which used marker detection to determine the robot's position and orientation in the arena.
- ➔ The electrical system connected the WeMos D1 microcontroller to the motor controller, ultrasonic sensors, OLED display, and battery supply. The ultrasonic sensors provided short range distance measurements - left, right, and front. The OLED display was used during testing to show live debugging data, including sensor readings and navigation values. Power was distributed directly through bundled jumper wires instead of a breadboard to improve durability during motion and repeated handling.
- ➔ Our software system was implemented directly in the Arduino IDE using C++. MQTT communication provided by the sample code provided the robot's coordinate data from

the Smart Arena network. The navigation algorithm used this position data to calculate the robot's distance and heading relative to each target. Sensor readings were then used with motor-control functions to adjust the robot's path and avoid obstacles.

➔ Our final design integrated these subsystems to establish our pathfinding and obstacle avoidance logic.

Project Objectives (Balakrishnan, Gerdek)

➔ The primary objective of this project was to develop a reliable autonomous robot that could complete a client-defined navigation task in the Smart Arena with minimal human intervention;. The robot needed to interpret its real time position, determine an efficient route to each assigned target, adjust its heading during motion, and respond to nearby obstacles without remote control. In addition to completing the course, the design needed to remain stable, repeatable, and easy to maintain during testing. This meant we had to prioritize secure mounting, organized wiring, accurate sensor placement, clear debugging feedback, and modular code that could be adjusted easily during trials.

Design Requirements (Balakrishnan)

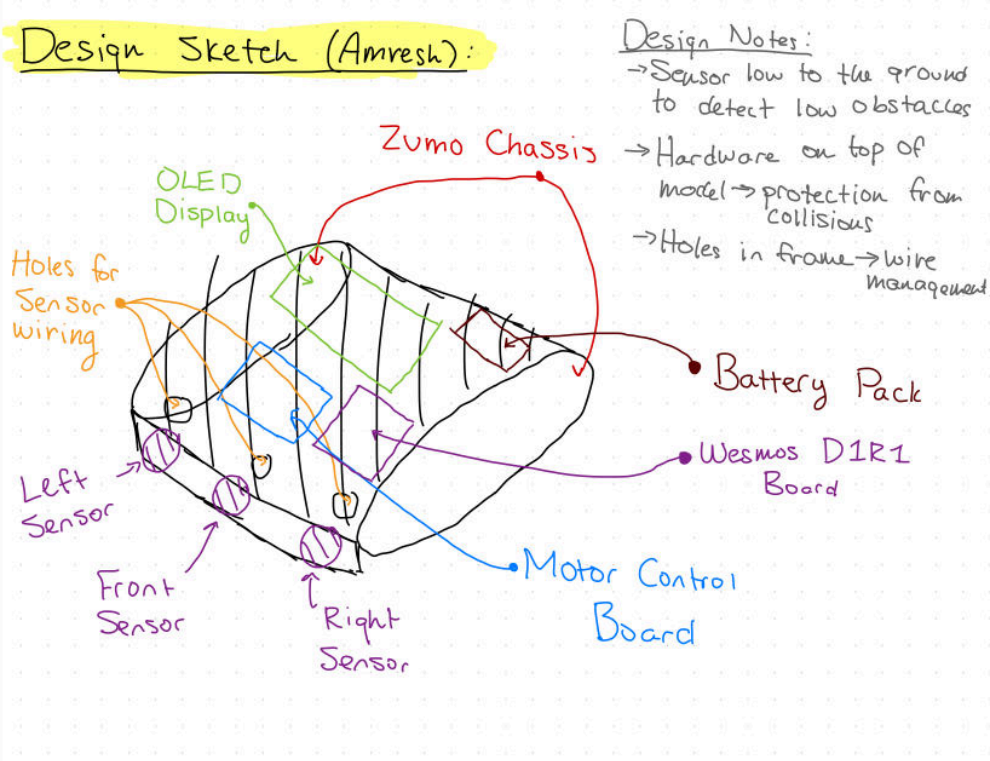
➔ The project outlined numerous design requirements that our final system had to meet, spanning mechanical, electrical, software, and performance. Mechanically, the robot needed a custom 3D-printed body that could securely mount all required components without glue or tape, including a bracket for two 4cm x 4cm ArUco markers positioned below the 6.5 inch height limit. The final design also needed to follow the 3D printing constraints: all printed parts had to stay within a 5-hour total print time, and each individual print job had to be completed within the 2 hour 50 minute class period, based on the Flashprint slicer estimate using the Adventurer 3 model. Electrically, the final circuit could not use a breadboard and instead needed a reliable power bus, color-coded jumper wires, and a slide switch to prevent short circuiting the Arduino board. The robot also needed at least three ultrasonic sensors, an OLED display, a WeMos D1 microcontroller, a motor controller, and ArUco based camera tracking. For software,the robot needed to utilize the Week 7 MQTT baseline code, be programmed as an Arduino sketch, include an algorithm flowchart, and use real time coordinate data for path planning and obstacle avoidance rather than a time based movement. Lastly, the robot had to reach four assigned target locations in order, stop for two seconds after each target, avoid collisions, and complete the task within the three minute time limit.

Concept Design & Design Alternatives Considered (Balakrishnan, Gerdek)

➔ During the concept design stage, our team considered different ways to organize the robot's mechanical layout, sensor placement, ArUco marker support, wiring, and movement strategy. The purpose of this stage was to compare possible designs and select the option that best satisfied the project requirements. The final robot needed to use the Smart Arena camera system, ArUco markers, at least three ultrasonic sensors, an OLED display, and a stable robot body without using glue or tape. It also needed to safely reach target locations while avoiding obstacles.

Initial Concept (Figure below):

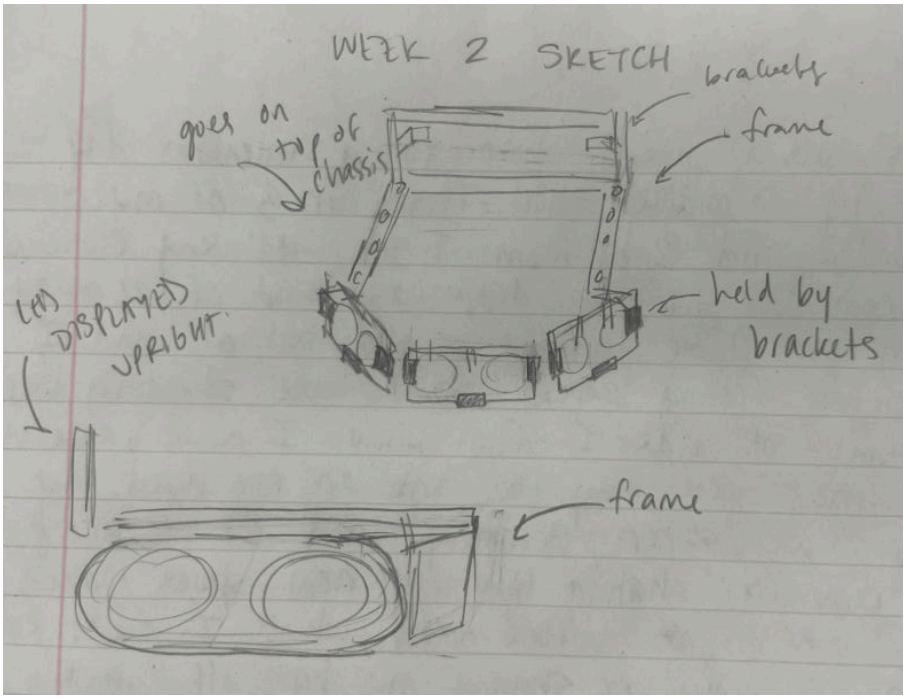
➔ Our first concept focused on arranging the main components on top of the Zumo chassis. This included the WeMos board, motor controller board, battery pack, OLED display, ultrasonic sensors, and the ArUco marker support. The early sketches helped us visualize where each component could fit and how the wiring would be routed. The main goal was to keep the design compact while still leaving enough space for the sensors and electronics.



➔ A major drawback of our initial design was the orientation of our ultrasonic sensors. By placing the three sensors all over the front surface, we were only measuring the front values. This was not only a redundancy, but we also were unable to detect collisions on the left and right of our robot, causing our obstacle avoidance to fail. Additionally, the location of the OLED display made it difficult to read during live debugging. In our later design, we aimed to change the location of the right and left ultrasonic sensors to the sides of the robot and move the OLED display to the back.

Final Design:

➔ After considering the changes from our initial design, we also needed to implement a mounting system for the ArUco marker. The marker had to be clearly visible from above and securely supported during motion. Initially, we created a simple, slender marker, but the final design favored a more stable 3D-printed stand that could hold two ArUco markers. This design was selected because it improved camera visibility and helped prevent the markers from shifting while the robot moved.



Movement and Tuning Alternatives:

- ➔ Our team tested several movement strategies to evaluate our new design. We utilized simple test sketches to upload simple instructions to calibrate the wheel speeds relative to each other. Some turning methods caused the robot to rotate sharply to one direction, while others did not turn enough to avoid obstacles or correct the robot's direction. The final movement strategy used separate functions for forward motion, stopping, left correction, right correction, and 90 degree turns. This made the code easier to run and allowed the robot to make smaller corrections while navigating.

Electrical Connection Alternatives:

- ➔ Our team also considered soldering the circuit, but decided to use wire connectors and organized cable management instead. Soldering was a viable option to create stronger pin connections, but would make modifying the circuit during testing impossible. Since our robot required repeated testing and adjustment, write connections were selected because they allow for easier troubleshooting and component changes. This approach also avoided using a breadboard, which was not allowed under the final design requirements.

Evaluation of Design Concepts

Quantifying Design Choice (Balakrishnan & Alteveer):

- ➔ We used a pairwise comparison chart to determine our design objectives, and establish which objectives were most important in our final design. Then, we discussed as a group which of our designs best met these requirements and chose to move forward with that design for our final project.
- ➔ Our final concept was selected because it balanced stability, sensor visibility,obstacle detection, wiring organization, and ease of assembly. Compared to the earlier concepts the final design better supported the required electronics and allowed the robot to use both MQTT camera feedback and ultrasonic sensor feedback for navigation.

Robot Final Design

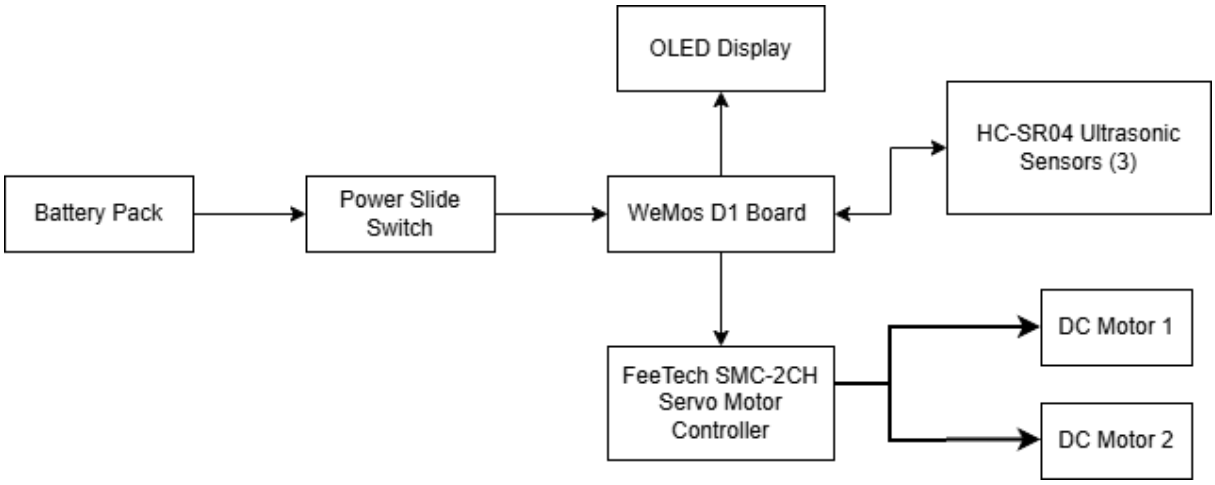
Pairwise Matrix	vs. Mech. Functionality	vs. Mech. Constraints	vs. Print Time	vs. Durability	vs. Weight Dist.	vs. Wire Org.	vs. Aesthetic	Score	Rank
Mech. Functionality	—	1	1	1	1	1	1	6	1st
Mech. Constraints Met	0	—	1	1	1	1	1	5	2nd
Print Time	0	0	—	1	1	1	1	4	3rd
Durability	0	0	0	—	1	1	1	3	4th
Weight Distribution	0	0	0	0	—	1	1	2	5th
Wire Organization	0	0	0	0	0	—	1	1	6th
Aesthetic Appeal	0	0	0	0	0	0	—	0	7th

Design: Constraints (C) Objectives (O)	Priority	Design 1 Score	Weighted Design 1 Score	Design 2 Score	Weighted Design 2 Score
C: Safety	4	0	0	0	0
O: Obstacle Avoidance	5	40	200	95	475
O: Navigation Accuracy	3	70	210	90	270
O: Stability	2	70	140	90	180
O: Ease of Assembly	1	70	70	80	80
Total			**620**		**1005**

Final Design Description (Alteveer)

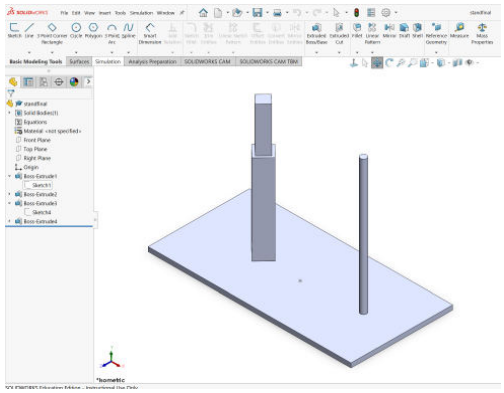
System Architecture – Input & Output Flow (Alteveer):

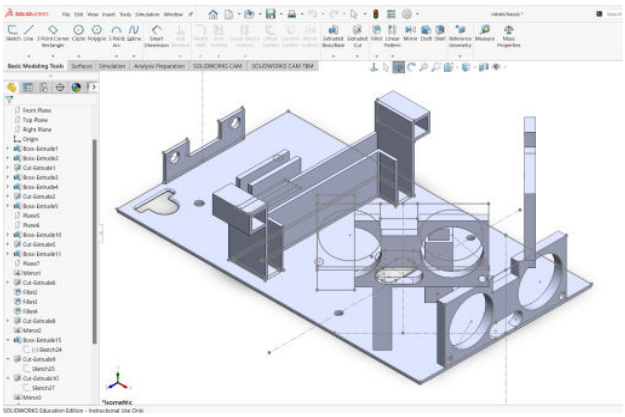
➔ Our final design incorporated three ultrasonic sensors, one OLED display, one Arduino WeMos D1 board, one “AA” battery pack consisting of 4 batteries, a Zumo Chassis, microcontroller board, and two motors. The battery pack provides the input for the arduino board, in which pin connections provide energy to all of these components. The sensor information is then given as the input to both the OLED display, and the microcontroller, giving the motors information. Based on our code, which was uploaded to the Arduino board, the sensor information is given to our microcontroller, which then passes on that input to the motors, turning the tread of our Robot.



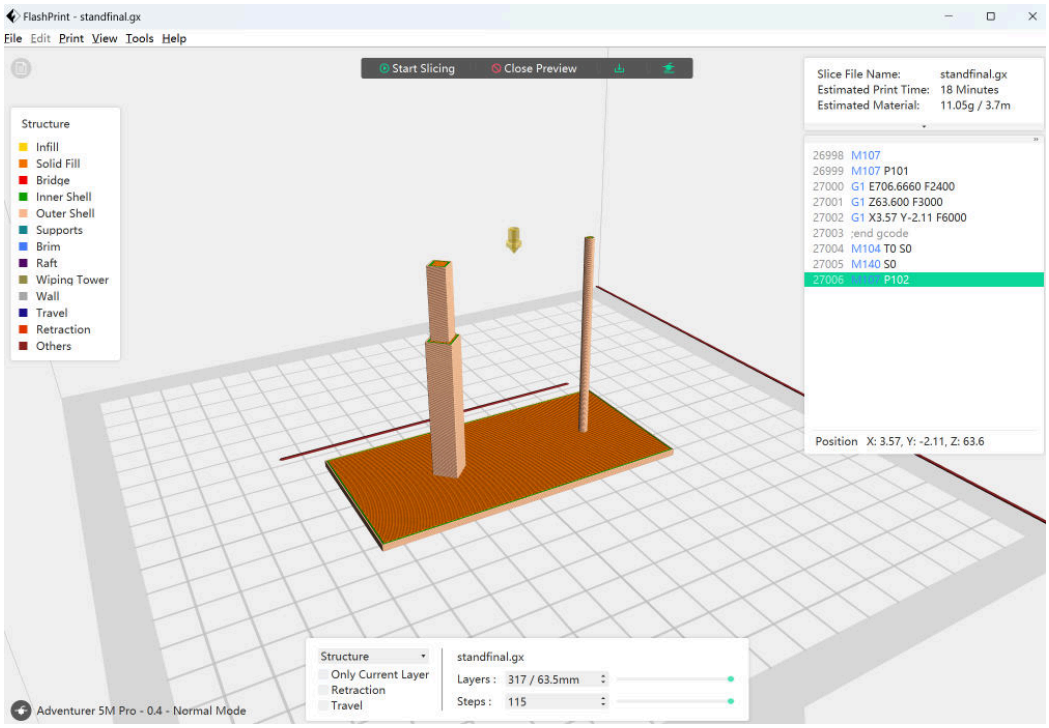
Mechanical Design (Alteveer):

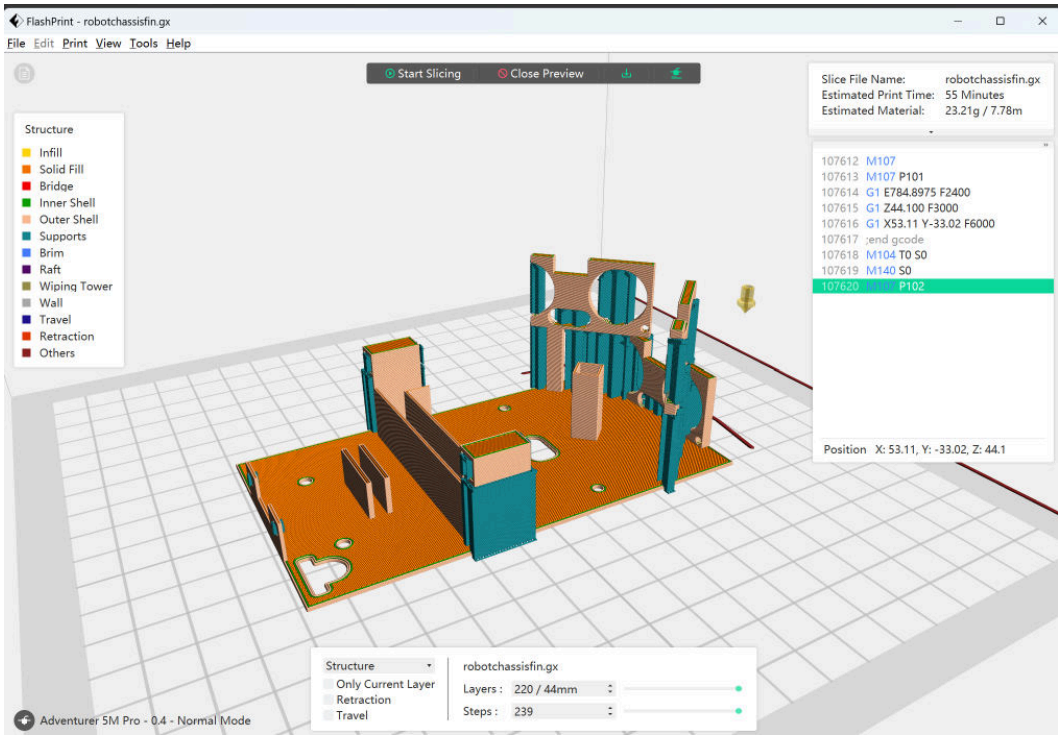
➔ For our mechanical design, we used two main parts. Our main Robot chassis uses M1.6 and M2 fasteners (nuts and bolts) to secure on top of the Zumo Chassis. Our second 3D printed part snugly fits as a flat stand on which our two ArCuo markers can be clearly displayed. M1.6 and M2 fasteners are also used to secure our ultrasonic sensors to the 3D printed chassis; displayed below is our working Assembly view for the finished parts.





➔ The figures below are the two 3D printed parts. Our ArCuo marker stand failed printing several times with the design displayed in the assembly. After evaluation, it was clear the printing temperature was too hot in between layers, resulting in a warped, melted peg for the stand. This inhibited the functionality of the stand, as the tolerance was very little to ensure a snug, secure fit that wouldn't easily budge. No supports were automatically generated for our stand design when sliced, so as a solution we added a "sacrificial" column. This column was designed to be extremely fragile, easily breakable, the same height as the tallest point of our design, and far away from the main functional column of the stand. This gives the printer part time to cool down in between layers, preventing overheating and letting the layers cool slightly before moving to the next layer. This ensures solidity of the main column, and the fragility of the sacrificial column means it doesn't impact the final design.


FlashPrint - standfinal.gx
File Edit Print View Tools Help
Start Slicing
Close Preview
Slice File Name: standfinal.gx
Estimated Print Time: 18 Minutes
Estimated Material: 11.05g / 3.7m
Structure
Infill
Solid Fill
Bridge
Inner Shell
Outer Shell
Supports
Brim
Raft
Wiping Tower
Wall
Travel
Retraction
Others
26998 M107
26999 M107 P101
27000 G1 E706.6660 F2400
27001 G1 Z63.600 F3000
27002 G1 X3.57 Y-2.11 F6000
27003 ;end gcode
27004 M104 T0 S0
27005 M140 S0
27006 M[illegible] P102
Position X: 3.57, Y: -2.11, Z: 63.6
Structure
Only Current Layer
Retraction
Travel
standfinal.gx
Layers : 317 / 63.5mm
Steps : 115
Adventurer 5M Pro - 0.4 - Normal Mode


FlashPrint - robotchassisfin.gx
File Edit Print View Tools Help
Start Slicing
Close Preview
Slice File Name: robotchassisfin.gx
Estimated Print Time: 55 Minutes
Estimated Material: 23.21g / 7.78m
Structure
Infill
Solid Fill
Bridge
Inner Shell
Outer Shell
Supports
Brim
Raft
Wiping Tower
Wall
Travel
Retraction
Others
107612 M107
107613 M107 P101
107614 G1 E784.8975 F2400
107615 G1 Z44.100 F3000
107616 G1 X53.11 Y-33.02 F6000
107617 ;end gcode
107618 M104 T0 S0
107619 M140 S0
107620 M[illegible] P102
Position X: 53.11, Y: -33.02, Z: 44.1
Structure
Only Current Layer
Retraction
Travel
robotchassisfin.gx
Layers : 220 / 44mm
Steps : 239
Adventurer 5M Pro - 0.4 - Normal Mode

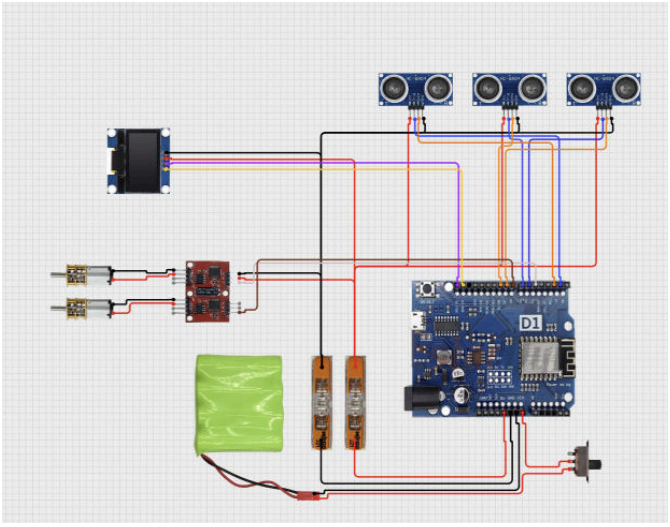
➔ For our final design we used zip ties as fasteners for a bulk of the prototyping process, as it was much more efficient to remove parts that were zip tied to the chassis rather than individually unscrewing each mechanical fastener. Zipties were not allowed in the final design however, so we made sure to include holes that would fit the tolerance of M1.6 and M2 bolts and nuts.

Electrical & Wiring Design (Alteveer):

➔ We opted to use a lever style wire connector instead of soldering for a variety of reasons. Solder, although effective and a useful tool set, was unreliable from what we observed. After consulting both TA's and other groups, there seemed to be a lot of issues with solder sticking to the robot and staying secure through test runs. To ensure wire connections staying as reliable as possible, we used a lever style wire connector similar to the one in right figure.



➔ As a part of our design requirements, our wiring diagram includes a slide switch as well, which was also a part of our final robot design. This helped combat earlier issues we had with short circuiting the Arduino WeMos board, which was a problem we encountered in our earlier prototyping.



➔ In terms of pin functionality, the figure below describes the wire connections depicted on the wiring diagram above. Motor pins needed to have PWM capabilities, and we strategically chose D3 and D6 for each motor's signal. D1 and D2 pins were the only pins with SDA and SCL capabilities, and were used for the OLED board. The positioning of all the other PWM pins for other components were found through trial and error, as we tried many different combinations before settling on the most efficient combination of pins. Duplication/reuse of any PWM pins was avoided at all costs, as it would have significantly interfered with wiring circuit functionality.

WEMOS BOARD PIN	PIN FUNCTION	COMPONENT PIN CONNECTION
D1	PWM / I/O, SCL	OLED: SCL
D2	PWM / I/O, SDA	OLED: SDA
D3	PWM / I/O, 10k Pull-up	Motor 1: Signal
D6	PWM / I/O, MISO	Motor 2: Signal
D0	PWM / I/O	Sensor: Echo
D4	PWM / I/O, 10k Pull-up	Sensor: Trig
D5	PWM / I/O, SCK	Sensor: Trig
TX	TXD, Transmit for Serial Data, can be used as PWM	Sensor: Trig
D7	PWM / I/O, MOSI	Sensor: Echo
D8	PWM / I/O, 10k Pull-up	Sensor: Echo

Software Design & Coding (Balakrishnan):

➔ All of our robot software was written in the Arduino IDE using C++ and uploaded to the WeMos D1 microcontroller. The final program combined MQTT coordinate feedback, ultrasonic obstacle detection, motor control, and OLED debugging/display. The provided Week 7 MQTT baseline code was used for the Wifi information and Smart Arena communication, while we added the target navigation, obstacle avoidance, motor movement, display logic, and imported the math.h library.

➔ Our program assigned the left motor to pin D3, right motor to pin D6, middle ultrasonic sensor to pin D5 and D8, left ultrasonic sensor to pins TX and D0, and right ultrasonic sensor to D4 and D7. The OLED display used I2C communication with SDA on D2 and SCL on D1. These pin assignments allowed the Arduino board to read sensor data, display debugging information, and control both motors at the same time.

➔ Our gram used the libraries MQTT.h, Wire.h, SSD1306Wire.h, Ultrasonic.h, Servo.h, and math.h. The MQTT library received the robot's position and orientation room, the Smart Arena camera system. The Wire.h and SSD1306Wire.h libraries controlled the OLEd display through I2C communication. The Ultranoic.h library read distance values from the three ultrasonic sensors and the Servo.h library controlled the left and right motors. The math.h library was used for all of the navigation calculations (dx,dy, desired_angle), including square root and inverse tangent functions. Below, we included the main logic of our code:

```
distance_middle = ultrasonic_middle.read(CM);
distance_right  = ultrasonic_right.read(CM);
distance_left   = ultrasonic_left.read(CM);

float dx = x_targets[target_index] - x_robot;
float dy = y_targets[target_index] - y_robot;
float distance_to_target = sqrt(dx * dx + dy * dy);

float desired_angle = atan2(dy, dx) * 180 / PI;
float angle_error = desired_angle - z_ang_robot;

if (distance_to_target < 100) {
   stop();
   target_index++;
   delay(2000);
}

if (distance_middle < 16) {
   if (distance_left < distance_right) {
      turn_right();
   } else {
      turn_left();
   }
}

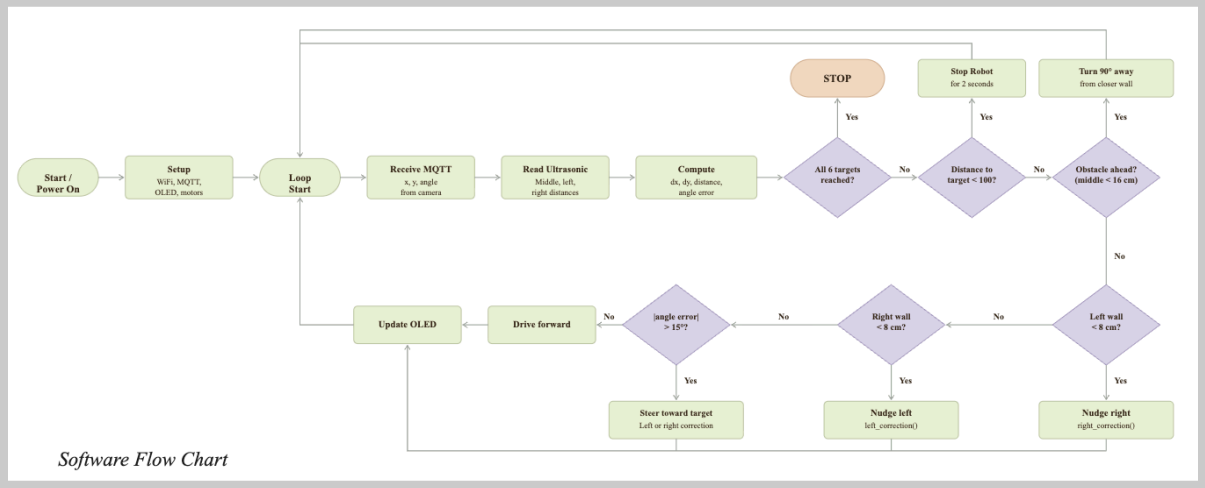
if (angle_error > 15) {
   left_correction();
} else if (angle_error < -15) {
   right_correction();
} else {
   forward();
}
```

➔ Our navigation algorithm used an array of x and y coordinate checkpoints. During each loop, the robot compared its current MQTT position, x_robot and y_robot to the current checkpoint. The program calculated the x_distance and y_distance and assigned the value to dx and dy respectively. Then, it calculated the total distance to the checkpoint using the

sqrt function from the math.h library to implement the pythagorean theorem (distance_to_target = sqrt(dx*dx + dy*dy). The robot also calculates the angle from the same mat.h library with atan2 (desired_angle = atan2(dy,dx) * 180/PI).

➔ The desired angle was compared to the camera-measured robot angle z_ang_robot, to find the angle_error. If the angle error was not negligibly (plus or minus 15 degrees), the robot would make a correction to orient the robot towards the checkpoint. If the angle_error was acceptable, the robot would proceed straight forward.

➔ The obstacle avoidance algorithm used the three ultrasonic sensor readings. If the front sensor detected an object closer than 16 cm, the robot compared the left and right sensor distances and turned toward the side with more open space. If the left sensor detected an object closer than 8 cm, the robot corrected right. If the right sensor detects an object closer than 8 cm, the robot corrects the left. This allowed the robot to avoid collisions from all directions. For a comprehensive look at our algorithm, please refer to the software flowchart below.



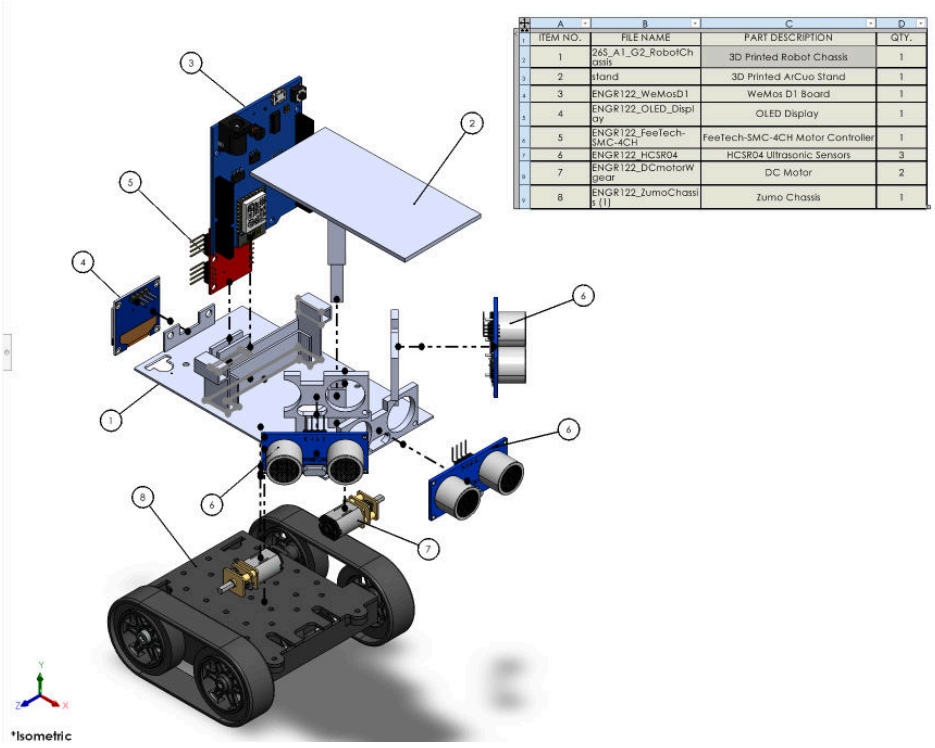
➔ The motor control code to adjust the direction of the robot was placed within functions ahead of the main loop. By implementing principles of object oriented programming, we were able to quickly change our code when debugging and increase the readability of our overall code base. Our defined functions include: forward(), stop(), left_correction(), right_correction(), turn_left(), turn_right(), and reverse(). This stop() function wrote 90 to both motors, while the movement functions used values above or below 90 to control direction and speed. A value above 90 corresponds to a forward motor action and a value

below 90 moves the motor backwards. Also, our turning and correction functions featured a timing delay to prolong the correction to meet our function requirements. For example, a right_correction() had a very short delay to turn the robot a few degrees, compared to the right_turn() function which turned the robot 90 degrees.

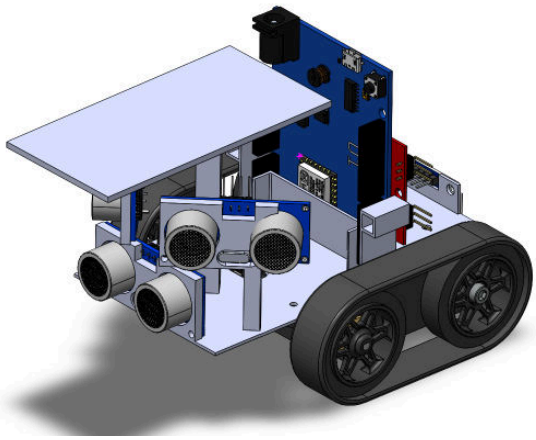
- ➔ Our team used the OLED display as a critical component during the debugging process and testing. Each loop printed the middle, left, and right ultrasonic sensor readings. This helped verify whether sensors were reading correctly and helped us identify faulty hardware within our system.
- ➔ When the robot reached within 100 mm/10cm of a checkpoint, the program stopped both motors, increased the target index, and waited two seconds before continuing. After all checkpoints were completed, the robot stopped.

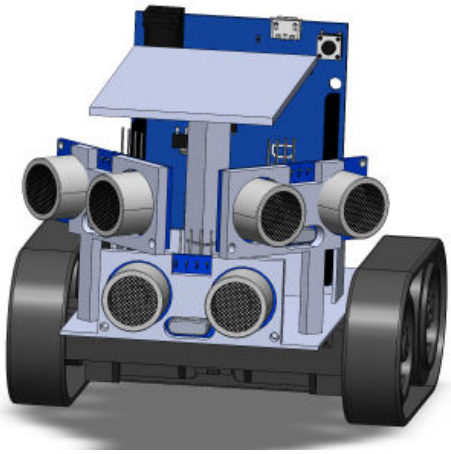
Assembly Instructions & Prototyping Specifications (Alteveer):

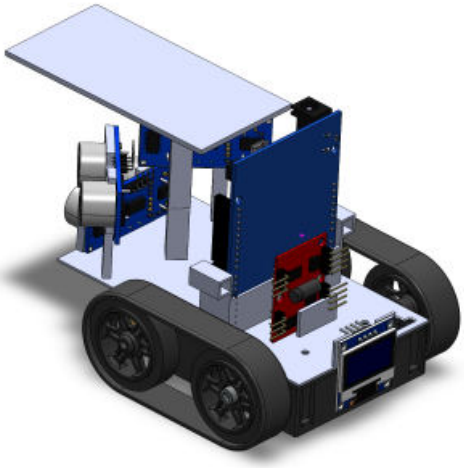
- ➔ The robot assembly was documented in Solidworks using a full assembly model, an exploded view, and a bill of materials. The custom 3-D printed chassis plate was fastened to the Zumo chassis using M1.6 and M2 nuts and bolts. The WeMos D1 board, motor controller, battery pack, OLED display, and three ultrasonic sensors were then mounted to the chassis in their assigned locations. The separate 3D-printed ArUco marker stand was attached above the chassis so the two markers remained stable and visible to the overhead camera. After the mechanical assembly was complete, the sensors, motors, OLED display, slide switch, and jumper cables were connected according to the final writing diagram. Below are detailed views of the Solidworks assembly.

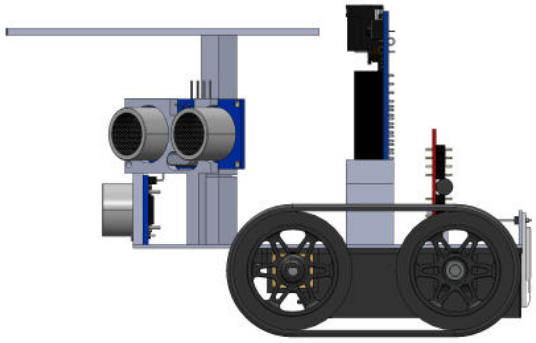


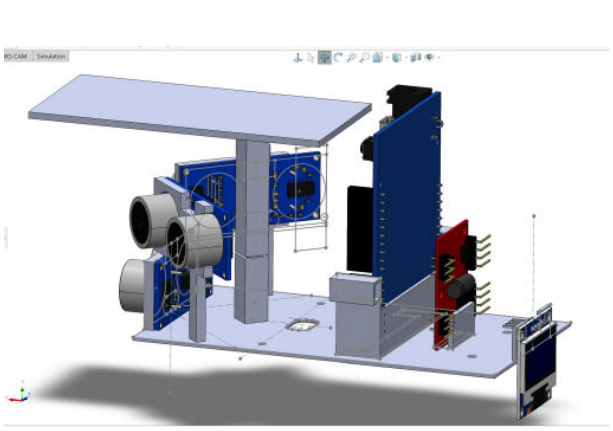
ITEM NO.	FILE NAME	PART DESCRIPTION	QTY.
1	26S_A1_G2_RobotChassis	3D Printed Robot Chassis	1
2	Stand	3D Printed ArCuo Stand	1
3	ENGR122_WeMosD1	WeMos D1 Board	1
4	ENGR122_OLED_Display	OLED Display	1
5	ENGR122_FeeTech-SMC-4CH	FeeTech-SMC-4CH Motor Controller	1
6	ENGR122_HCSR04	HCSR04 Ultrasonic Sensors	3
7	ENGR122_DCmotorWgear	DC Motor	2
8	ENGR122_ZumoChassis (1)	Zumo Chassis	1

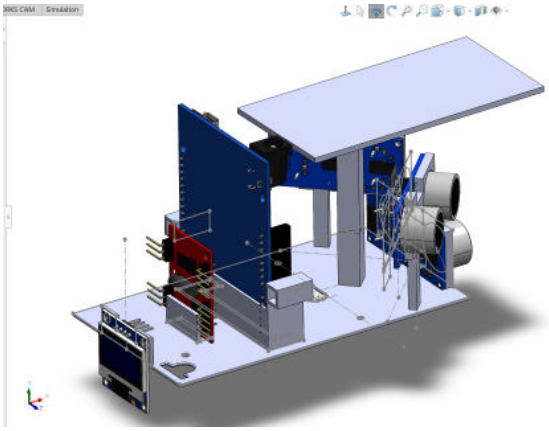






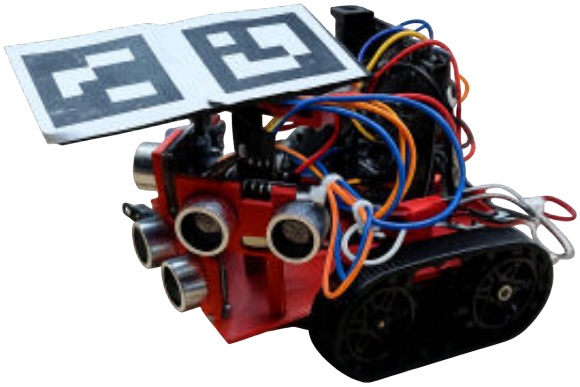




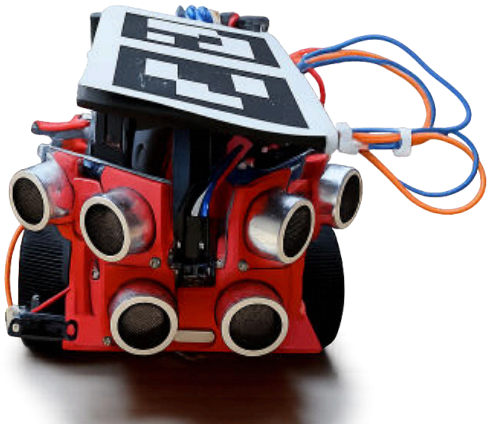


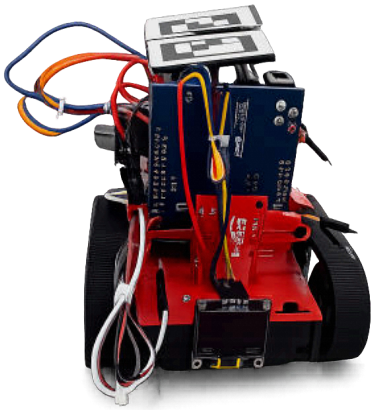
Final Assembly (Balakrishnan)

➔ Below, we have included images of our final assembly that was utilized in the Smart Arena challenge. Our final system incorporates all of our previous designs and is representative of the iterations we made throughout the design process.









Performance (Balakrishnan)

➔ During the week 13 testing, our robot successfully reached all four assigned targets in the required order, completed the course in 52 seconds, and had no collisions. These key performance indicators satisfied the main objectives of the challenge and were well under the time requirement of three minutes. Earlier issues including a short-circuited Arduino board, faulty jumper wires, and faulty ultrasonic sensors, were resolved before testing and all of our robots subsystems and hardware worked as expected

Concluding Remarks (Balakrishnan)

➔ Overall, our final project met all the requirements by combining mechanical, electrical, and software design. Ava led the mechanical design, wiring, Solidworks graphics, and related report sections; Amresh led the software development, pathfinding and obstacle avoidance algorithms, corresponding report sections, and completed most of the final report; and Abuzer completed most of the presentation and some parts of the final report. Although the final design performed successfully, future improvements could include smoother turning behavior, additional sensors for better obstacle avoidance, and use of the onboard memory to remember specific obstacles. All things considered, our group learned a lot about the design and debugging process and are extremely proud of our final project!

Appendix (Balakrishnan)

Full Arduino Sketch Code:

```
/*==========================================================================
============================================================

 * Ver.1 – 03/02/2026 SES IDEAs Program

 * 2026S ENGR122 Sample Code for Week 7 Activity & Final Project

 * Camera Sensor–Based Position Feedback using MQTT Network

 *

 * This sketch is provided as a baseline for the Week 7 lab activity and Final Project

 * Week7 Activity:

 *  -Based on this baseline code, students must complete the program to display the robot's position on the OLED display.

 * Final  Project:

 *  -Using the ultrasonic sensor, students must design an Obstacle Avoidance logic.

 *  -Using camera-sensor data received through the MQTT network, students must implement a Path-Planning algorithm.

 *  -Students must integrate these algorithms to control the two motors so that the robot can reach designated targets.

 *  -The OLED display should be used for debugging purposes.
```

```
*==========================================================================
==========================================================*/

 //Names: Amresh, Ava, Abuzer
 //Pledge: We pledge our honor that we have abided by the Stevens Honor System.

//(NOTE!) Include all required libraries here
#include "MQTT.h"
#include <Wire.h>
#include <SSD1306Wire.h>
#include <Ultrasonic.h>
#include <Servo.h>
#include <math.h>



//(NOTE!) You may create any additional objects needed for OLED, Sensors, or Motors here

//motor pins --> don't change
#define motor_left_pin D3
#define motor_right_pin D6

// object creation for motors
Servo motor_left;
Servo motor_right;

//object creation for ultrasonic sensors
Ultrasonic ultrasonic_middle(D5, D8);
Ultrasonic ultrasonic_right(D4, D7);
```

```
Ultrasonic ultrasonic_left(TX, D0);

//object creation for OLED display
SSD1306Wire display(0x3C, D2, D1);

//target coordinates w/ 4 midpoints placed in
const int x_targets[] = {689, 1566, 2000, 2133, 545, 155};//Sample x-coordinates for targets
const int y_targets[] = {677, 139, 100, 623, 410, 138};//Sample y-coordinates for targets

// target reached counter
int target_index = 0;

//_____________________________________________________________________________

//MQTT Setting --> DO NOT TOUCH
const char* mqtt_server   = "192.168.0.2";
const char* mqtt_username = "user";
const char* mqtt_password = "Stevens1870";
const int mqtt_port = 1883;

//MQTT Topic
const String topic     = "EAS011_North";       //North Arena
//const String topic   = "EAS011_South";       //South Arena
//const String topic = "SmartArena";    //Integration Lab

//WiFi Setting
const char* ssid       = "TP-Link_8FFD";       //North Arena Check the router on the TA desk.
const char* password   = "68287078";           //North Arena Check the router on the TA desk.
```

```
//const char* ssid     = "TP-Link_9402";         //South Arena Check the router on the TA desk.
//const char* password = "77556578";             //South Arena Check the router on the TA desk.
//const char* ssid = "TP-Link_7310";    //Integration Lab
//const char* password = "74952359";    //Integration Lab
//_____________________________________________________________________________

//target values --> from ArDuco markers
float x_robot, y_robot, z_ang_robot;

//(NOTE!) You may add additional variables here as needed, but carefully consider variable scope (global vs. local)

//values from ultrasonic sensors

int distance_middle;
int distance_right;
int distance_left;

//setup --> runs every robot start

void setup() {
   Wire.begin();
   delay(25);
   wifi_mqtt_init();
   mqtt_clean();
   delay(500);
   display.init();
   display.flipScreenVertically();
   motor_left.attach(motor_left_pin);
```

```
    motor_right.attach(motor_right_pin);
    motor_left.write(90);
    motor_right.write(90);
    delay(2000);

}

// move robot forward
void forward(){
    motor_left.write(110);
    motor_right.write(109);
}

// stop robot --> cut both motors
void stop(){
    motor_left.write(90);
    motor_right.write(90);
}

// adjust robot orientation to the left
void left_correction(){
    motor_left.write(107);
    motor_right.write(120);
}

//adjust robot orientation to the right
void right_correction(){
    motor_left.write(120);
```

```
    motor_right.write(107);
}

// turn robot 90 degrees to the left
void turn_left(){
    motor_left.write(70);
    motor_right.write(110);
    delay(720);
}

// turn robot 90 degrees to the right
void turn_right(){
    motor_left.write(110);
    motor_right.write(70);
    delay(720);
}

// reverse the robot
void reverse(){
  motor_left.write(60);
  motor_right.write(60);
}

//used to filter out misreads from the robot --> recommended by office hours TA
unsigned long lastAction = 0;       // Timer for obstacle maneuvers
const unsigned long actionDelay = 300; // Duration for reverse/turn in ms

//loop --> runs continuously when robot is powered on
```

```
void loop() {
    //MQTT network code --> provided by baseline code
    unsigned long now = millis();
    mqtt_rebound();
    client.loop();

    // 3 ultrasonic sensor readings
    distance_middle = ultrasonic_middle.read(CM);
    delay(10);
    distance_right  = ultrasonic_right.read(CM);
    delay(10);
    distance_left   = ultrasonic_left.read(CM);
    delay(10);

    // robot distance calculations
    float dx = x_targets[target_index] - x_robot;   //distance between robot and target x-axis
    float dy = y_targets[target_index] - y_robot;   //distance between robot and target y-axis
    float distance_to_target = sqrt(dx*dx + dy*dy); //resultant vector produced by pythagorean theorem

    //robot angle calculations
    float desired_angle = atan2(dy, dx) * 180 / PI; //angle to point robot at target
    float angle_error = desired_angle - z_ang_robot; //current angle error

    // fit the robot angle between 180
    if(angle_error > 180) angle_error -= 360;
    if(angle_error < -180) angle_error += 360;

    // when all targets reached --> stop the robot
```

```
    if(target_index >= 6){
        stop();
        delay(5000);
    }

    // if not all targets have been reached --> run this loop
    if(target_index < 6){

    // when robot close to target, stop the robot
    if(distance_to_target < 100){
            stop();
            target_index++;
            delay(2000);
            lastAction = now;
        }

    //Collision Logic
    if(distance_middle < 16){
        if(distance_left < distance_right){
            turn_right();
            delay(10);
        }
        else {
            turn_left();
            delay(10);
    }
}
    if(distance_left < 8){
```

```
    right_correction();
  }
  if(distance_right < 8){
    left_correction();
  }

  // Path finding logic
  if(angle_error > 15){
    left_correction();
  }
  else if(angle_error < -15){
    right_correction();
  }
  else{
    // if no correction needed --> go straight
    forward();
  }

}
  //print to OLED display --> sensor data and Angle error
  display.setFont(ArialMT_Plain_10);
  display.clear();
  display.drawString(0, 0, "Ultra_Middle: " + String(distance_middle));
  display.drawString(0, 15, "Ultra_Left: " + String(distance_left));
  display.drawString(0, 30, "Ultra_Right: " + String(distance_right));
  display.drawString(0, 45, "Angle_Error: " + String(angle_error));
  display.display();
```

```
// MQTT settings --> provided by the baseline code
    mqtt_clean();
    while (millis() - now < 100){delay(5);}
}
```